TUHH
Hamburg
University of
Technology

ICS
Institute
of Control
Systems

Project work

Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning

Author:
Hiba Sikander

Supervisor: Jan Kaiser, M.Sc.
Jannis Lübsen, M.Sc.
Examiner: Prof. Dr.-Ing Annika Eichler

February 23, 2026

ICS
Institute
of Control
Systems

Project Work
for Ms. Hiba Sikander

P — XX XX-XX —
Student ID: 610416

Study Course: L2353

Title:
Bayesian Optimisation with Differentiable Simulated Priors for Particle Accelerator Tuning

Project Description:
The tuning of particle accelerators is a time-consuming and difficult task that continues to be performed manually in many cases, reducing the scientific output of some of the most advanced but also costly research facilities in the world. In the field of autonomous accelerator tuning, researchers aim to develop methods that assist human operators and thereby reduce tuning times and extend the experimental capabilities of accelerators. One of the most promising approaches to solve the autonomous accelerator tuning problem is the use of Bayesian Optimisation (BO), which is already successfully used in operations. [1]

However, the most interesting accelerator tuning tasks are characterised by their high dimensionality. Despite being more capable at dealing with many dimensions than other algorithms, BO struggles to solve optimisations problems with more than a few dozen dimensions. To solve this challenge, researchers have proposed to leverage domain knowledge by replacing the commonly used but uninformative zero mean prior with a model of the system under optimisation. So far neural networks have been used for this purpose as they are differentiable and fast to compute. [2] While this work has shown the advantages of using an informed neural network prior, neural networks come with the disadvantage that they require large amounts of data to be trained, and that outside of the training data domain, their behaviour is ill-defined. In recent developments, high-speed differentiable simulators such as Cheetah [3] have been proposed as an alternative for use as a prior in BO, with the key advantage that they do not require any training data, and are well-behaved throughout. [3]

The goal of this research project is to take the BO with Cheetah prior approach presented on a very simple example in [3], and implement it on the closer-to-real world as well as more complex transverse beam parameter tuning task at the ARES accelerator, which has already been extensively studied with other autonomous tuning methods. The developed implementation should then be evaluated according to previous evaluations, to establish how this improved variation of BO compares to other methods and whether extension to even more complex accelerator tuning tasks is a promising avenue for future research.

Tasks:

1. Implement BO with Cheetah prior for tuning the transverse beam parameters on a simulation of the ARES Experimental Area.
2. Evaluate the BO with Cheetah prior implementation in accordance with the evaluation in [4].
3. Evaluate timing and speed of BO with an uninformed prior vs. BO with a Cheetah informed prior.
4. Study performance under model uncertainties.
5. **(Optional depending on accelerator availability)** Evaluate the developed implementation on the real ARES accelerator in accordance with [4].
6. **(Optional if time allows)** Setup a general implementation of BO with Cheetah.

References:

[1] R. Roussel et al., "Bayesian optimization algorithms for accelerator physics", *Physical Review Accelerators and Beams*, vol. 27, 2024.

[2] T. Boltz et al., "Leveraging prior mean modules for faster bayesian optimization of particle accelerators", *Scientific Reports*, vol. 15, 2025.

[3] J. Kaiser, C. Xu, A. Eichler, and A. Santamaria Garcia, "Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations", *Physical Review Accelerators and Beams*, vol. 27, 2024.

[4] J. Kaiser et al., "Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning", vol. 14, 2024.

Supervisor: Jan Kaiser, M.Sc.
Jannis Lübsen, M.Sc.
Examiner: 1. Prof. Dr.-Ing Annika Eichler

Deutscher Titel:

Start Date: 17.11.2025
Due Date: 02.03.2026

February 23, 2026, Prof. Dr.-Ing Annika Eichler

Hereby I declare that I produced the present work myself only with the help of the indicated aids and sources.

Hamburg, February 23, 2026 Hiba Sikander

Abstract

Tuning particle accelerators is an expensive and time consuming challenge that causes limitations in the findings of advanced research facilities. Bayesian Optimization (BO) has shown potential to be a promising approach for autonomous tuning of accelerators, yet its effectiveness diminishes for higher dimensional problems. Recent studies have proposed replacing the zero mean prior in BO with physics informed prior models to leverage physics domain knowledge. The Cheetah framework, a high-speed differentiable simulator, was demonstrated as a physics informed BO prior on a simple two dimensional FODO lattice. This project work extends that approach to a real world based five dimensional problem now, which is the transverse beam parameter optimization at ARES Experimental Area involving three quadrupole magnets and two corrector magnets. This project implements a framework that incorporates Cheetah as the Gaussian Process mean function into the Xopt optimization toolset with trainable misalignment parameters that enable the prior to adjust when the simulator does not accurately represent the real system. Four optimization approaches are evaluated. Results from 10 trails show that physics informed BO performed significantly better than baseline techniques. Both Cheetah prior variants (matched and mismatched prior) achieve 98.4% improvement in beam error with mean absolute error of 0.006 mm, compared to standard BO with 88.9% and 0.046 mm, and Nelder-Mead with 83.7% and 0.067 mm. While standard BO only reaches 20% and Nelder-Mead 0%, both Cheetah variants managed to achieve a 100% success rate in achieving the desired threshold (40 μm). Moreover, the matched prior converged the fastest, requiring only 4 steps (median) compared to 14 for mismatched prior. These results validate the scalability of differentiable simulator priors simple and straightforward to realistic and complex accelerator tuning problems.

Contents

1 Introduction

1.1 Motivation

Particle accelerators are one of the most advanced scientific devices ever developed to perform innovative scientific experiments. The Deutsches Elektronen-Synchrotron (DESY) [1] located in Hamburg region hosts multiple top notch accelerator facilities such as European XFEL X-ray Laser and the PETRA III. These facilities accelerate charged particles, that enable researchers to study atomic level processes and structures with great precision. A crucial challenge for particle accelerators is the tuning process, which involves adjustment of many magnetic components to produce desired beam properties including position, size and divergence. As a specialized test facility for accelerators research and development, the ARES (Accelerator Research Experiment at SINBAD) linear accelerator at DESY allows the evaluation of innovative tuning techniques prior to deployment on even larger facilities.

Beam tuning is essentially an optimization task. In order to find exact configurations that minimize the deviation between the actual beam properties and the desired beam, we optimize the adjustable parameters (like magnet strengths and corrector/steering angles). However, the optimization is not that simple and perhaps contains a number of challenges:

1. **Expensive Evaluation:** It is impractical to conduct an extensive search since measurement of each beam property takes time and resources.
2. **Uncertainity:** There are a number of systematic errors and noises that might affect beam measurement.
3. **Black-box Nature:** The complex relationship between input parameters and beam characteristics cannot be directly observed analytically during operation.
4. **Unknown / Partial Observability:** Important parameters like characteristics of incoming beam and magnet misalignments are frequently either unknown or partially observed.

A traditional approach to accelerator tuning could be through skilled professionals who manually adjust parameters using their knowledge and experience. While being efficient, this method significantly relies on individual skill, takes a lot of time, is expensive and does not scale well to more complicated accelerator systems. Therefore, automated optimization methods are necessary for effective accelerator performance. Several optimization algorithms have been applied to particle accelerator tuning problems with their own unique advantages and disadvantages. Nelder-Mead Simplex is a gradient free optimization technique and one of the approaches that is widely used because of its simplicity and robustness [2]. However they do not create models that generalize across many optimization runs and usually need many function evaluations to converge. Another approach is the Reinforcement Learning (RL) technique that has shown remarkable performance [3]. RL agents can learn to tune accelerators in very few steps during deployment by training

neural network regulations in simulation. However, RL requires extensive training data as well as very careful reward function design. The trained policies could not generalize well to conditions outside of their training distribution. The approach of our interest for this project work is Bayesian Optimization (BO), which finds a middle ground among these approaches. BO creates a probabilistic surrogate model of the objective function, which is usually a Gaussian Process, that provides estimates of uncertainty as well as predictions. This makes it possible for the assessment points to wisely balance between both exploration and exploitation. BO is a desirable option for accelerator tuning since it excels at solving expensive black-box optimization problems.

Another key insight motivating this project work is that, since the physics of particle accelerators is already well understood now, We can map the passage of charged particles via magnets using a reliable model and then they travel through magnetic fields according to well established physics knowledge. Thus, all thanks to modern simulation library like Cheetah [4] that we can now predict beam behavior with high accuracy. The standard BO uses uninformed zero mean prior function in the Gaussian process model. This implies that the algorithm must learn the whole function landscape from scratch and have to begin with no prior knowledge of the intended objective function values. On the other hand, if the physics informed knowledge could be incorporated into the prior, this shall give advantage and optimization could already start with meaningful predictions that could focus its limited evaluation budget on identifying the differences between the model and the reality. The importance of physics informed prior for accelerator optimization has also been shown in the recent work [5] where the neural network surrogate models trained on the simulation data can perform as effective prior mean functions, significantly speeding up the convergence. However, training neural network surrogates requires very large datasets from simulations. So let's continue studying Bayesian Optimization with Cheetah informed prior.

1.2 Extension from Simple FODO to Complex ARES Problem

Cheetah has already been demonstrated as a physics informed prior for BO on a very simple and straightforward beam focusing FODO (Focus-Drift-Defocus-Drift) lattice [5]. A FODO cell is a fundamental building block made up of alternating quadrupole magnets for focusing and defocusing that are separated by drift spaces.

Since this optimization problem was two dimensional, thus it only involved strengths of two quadrupole magnets (k^{Q1} and k^{Q2}) with the objective of minimising beam size at target location, defined in equation 1.1. Here σ_x and σ_y are the horizontal and vertical beam sizes, considering only beam size (focusing) without beam position (centering), unlike ARES problem, refelecting the simpler nature of FODO demonstration.

$$mae = 0.5(|\sigma_x| + |\sigma_y|) \tag{1.1}$$

Moreover, although there were two drift spaces (d_1 and d_2), they were constrained to have same lengths between the quadrupoles, therefore only a single trainable hyperparameter was there, which was restricted to the interval [0.2, 1.0] m and was included in the physics

informed prior mean function. The results showed that BO with Cheetah prior converged about 15 steps, indicating better sample efficiency than Nelder Mead Simplex and standard BO with zero prior. Furthermore, the method worked even with a mismatched prior, the algorithm learned the correct values during GP model fitting and converged to the true optimum when the drift lengths in the prior model (0.5m) was different from the ground truth (0.7m).

While this validated the fundamental concept, the FODO problem is an oversimplified example which does not depict the complex nature of actual accelerator tuning operations. The two dimensional search space is quite easy to explore and the model reality mismatch was restricted to a single geometric parameter (drift length) instead of the various other sources of uncertainty found in actual accelerators.

This project extends the FODO proof of concept to a significantly more complex and realistic problem which is transverse beam tuning at the ARES Experimental Area at DESY. The extension contain following key advancements:

1. **Higher Dimensionality:** In contrast to two dimensional FODO problem, ARES problem possess five dimensional search space including two steering/corrector magnet angles (θ_{CV} and θ_{CH}) and three quadrupole strengths (k^{Q1}, k^{Q2}, k^{Q3}). The volume of the search space is greatly increased by around 2.5x in dimensionality, which makes it more difficult and raises the importance of informed prior.

2. **Realistic Lattice:** The ARES Experimental Area is a real accelerator section loaded from an actual lattice description file unlike the synthetic FODO cell.

3. **Objective Nature:** For Ares problem, the objective function combines four beam parameters into a single metric, including horizontal and vertical sizes (σ_x, σ_y) as well as the positions (μ_x, μ_y). This makes simultaneous optimization of beam centering and focusing.

4. **Trainable Quadrupole's Misalignments:** Under this project work, the use of quadrupole misalignments as the cause of model reality mismatch is an important step forward. The ARES prior includes 6 trainable misalignment hyperparameters (horizontal and vertical for each quadrupoles Q1, Q2 and Q3) which are limited to physically plausible intervals of $\pm 0.5mm$, in contrast to the FODO demonstration, where only a single hyperparameter (drift length) was learned.

1.3 Contribution

Thus the main contributions of this work are :

1. Demonstrating that Cheetah informed prior approach is scalable from 2D synthetic lattice to 5D real world accelerator tuning problem.

2. Implementing physics informed prior mean function for the ARES lattice being compatible with the Xopt Bayesian optimisation framework

3. Extending prior with six trainable misalignment parameters, enabling the physics model to adapt to real accelerator conditions during optimization

4. Containing a comprehensive comparison between standard BO, Nealder-Mead and physic-informed BO (matched and mismatched prior) across multiple trials.

2 Theory

In this chapter, the theoretical foundations of this project work are developed. We begin with Gaussian processes as probabilistic function approximators, present Bayesian optimization as a framework for sample efficient black-box optimization and then eventually look over the Cheetah simulator and the ARES accelerator where the work is used and how physics informed knowledge may be integrated through the prior mean function.

2.1 Gaussian Process

Gaussian process (GP) offers a robust framework for regression which allows uncertainty quantification along with predictions. GPs are well suited for modeling unknown objectives in optimization problems [6] because they define probability distributions directly over functions, unlike parametric models that learn a predefined set of weights. A Gaussian process is fully specified by its mean function $m(x)$ and covariance function (kernel) $k(x, x')$.

$$f(x) \sim \mathcal{GP}(m(x), k(x, x')) \tag{2.1}$$

Here equation 2.1 states that a function $f(x)$ is drawn from GP where the mean function $m(x)$ encodes previous expectations regarding the behavior of the function and the kernel $k(x, x')$ carries assumptions regarding smoothness and correlation between points. The kernel selection determines what kind of functions the GP considers probable. Because of its ability in modeling smooth, physically realistic functions without assuming infinite differentiability, Matérn-5/2 kernel is widely used in Bayesian Optimization [6]. The GP uses Bayes's rule (equation 2.2) to update from prior to posterior when we observe data. The posterior mean remains Gaussian, with mean and variance that reflects both our prior assumption and observed evidence. One of the important characteristics is that posterior uncertainty is low in areas with dense observations and high in areas with sparse data, this natural uncertainty quantification is what makes GPs useful for guiding optimization.

$$p(f|X, y) \propto p(y|f, X)p(f|X) \tag{2.2}$$

Where $p(f|X, y)$ is the posterior (Updated GP), $p(f|X)$ is prior (GP) and $p(y|f, X)$ is the likelihood of observed data.

2.2 Bayesian Optimization

Bayesian Optimization (BO) is a sequential method for determining the optimal value of expensive black-box functions [6]. It is especially effective when gradient information is not available and each function evaluation is expensive. Because of these characteristics, BO is ideally suited for accelerator tuning, where it has emerged as a cutting-edge method [7] [8]. The goal is to solve, shown in equation 2.3:

$$x^* = \arg\min_{x \in \mathcal{X}} f(x) \tag{2.3}$$

Here, x represents the control variables (like magnet settings), $\mathcal{X}$ is the search space and $f(x)$ is the quantity to be minimised (beam error in our case). BO aims to find x^* using as few function evaluation as possible. The key idea is to use a probabilistic surrogate model (usually a GP) to guide the search rather than evaluating the expensive function everywhere. As shown in Figure 2.1, the optimization process is iterative, We start with a modest initial set of observations, fit a GP model to the data and then choose the next point to evaluate using an acquisition function. Exploration (sampling when uncertainty is large) and exploitation (sampling where the predicted value is good) are balanced by the acquisition function. After evaluating the true objective at the selected point, we update the dataset and repeat until the evaluation budget is exhausted.

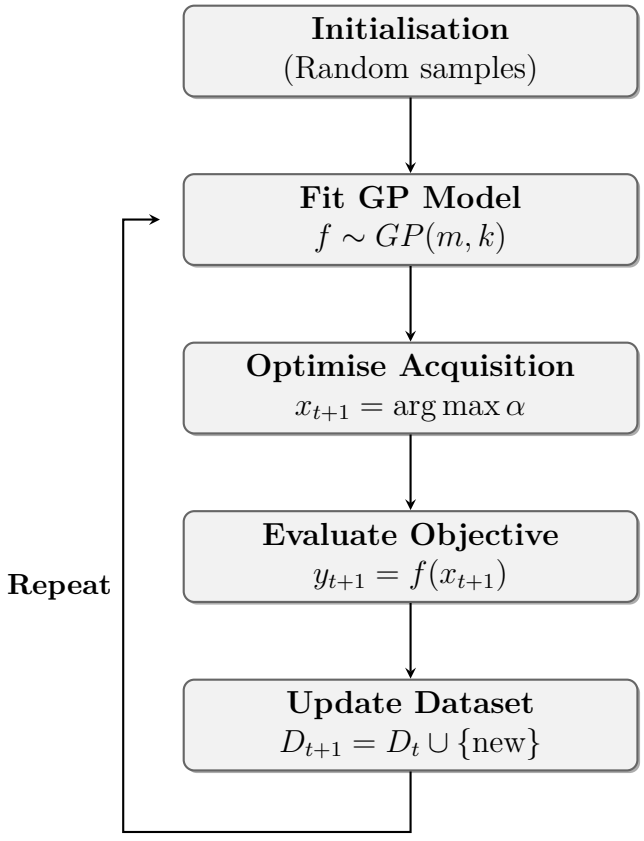

Figure 2.1: The Bayesian optimisation loop.

Acquisition functions guide where to sample next by balancing two competing goals, where one is exploring uncertain regions while the other is exploiting areas that look promising. Two widely used aquisitions funtions are Expected Improvement (EI) and Upper Confidence Bound (UCB).

- **Expected Improvement:** This acquisition function computes how much better do we expect a candidate point to be compared to our current best observation

$f(x^*)$, the expectation is taken over the GP's predictive distribution. EI naturally trades off exploration and exploitation wihtout any tuning parameter.

$$\alpha_{EI}(x) = \mathbb{E}[\max(0, f(x^*) - f(x))] \tag{2.4}$$

Where $f(x^*)$ is the best (lowest) observed value so far. The expectation averages over what the GP thinks might happen at x and the max ignores cases where things get worse by ensuring that only improvements are counted.

- **Upper Confidence Bound:** This acquisition function takes more direct approach by simply combinig the prredicted model and uncertainity.

$$\alpha_{UCB}(x) = -\mu(x) + \beta\sigma(x) \tag{2.5}$$

Where $\mu(x)$ and $\sigma(x)$ are the GP poterior mean and standard deviation respectively, and parameter β controls the exploration-exploitation tradeoff.

This project work, uses the Upper Confidence Bound (UCB) acquisition function (equation 2.5) with $\beta = 2.0$, following the approach used in the first demo over simple FODO problem [4] and common practice in accelator optimization [6].

2.3 Physics Informed Prior Mean Function

Before any data is observed, our belief about the objective is represented by the prior mean function $\mu(x)$. The standard practice is to use zero or constant mean, which assumes no prior knowledge. Physics models that approximate the system are often used in scientific applications and we have seen that incorporating this knowledge can significantly improve the performance [5]. To understand why, consider the GP posterior mean prediction at a new point. The GP learns to predict the difference between the prior mean and the actual function. These residuals are small and can be learned from fewer observations when the prior mean is accurate. Whereas when the prior is simply zero, the GP has to learn the entire function from scratch. This motivates using a physics informed prior mean function (equation 2.6):

$$m(x) = f_{physics}(x; \phi) \tag{2.6}$$

Where $f_{physics}$ is the physics model prediction, x being input to the optimization, which in our case are the five magnet settings required to be optimized (k_{Q1}, k_{Q2}, θ_{CV},k_{Q3}, θ_{CH}) and ϕ represents parameters of physics model that are uncertain or unknown like quadrupole misalignments, beam characteristics, drift lengths etc. In this project, it has been chosen to focus on quadrupole misalignments because they are a major source of model reality mismatch in ARES accelerator tuning. This benefits to improve sample efficiency and having meaningful predictions before observing data. In standard GP practice, as shown in equation 2.7, every time the GP model is fitted, it optimizes the kernel hyperparameters ω (length scales, signal variance, noise variance) by maximising the log marginal likelihood of observed data [9].

$$\omega^* = \arg\max_{\omega} \log p(y|X, \omega) \tag{2.7}$$

A key innovation in this project work is making the physics model parameters ϕ (in our case quadrupole misalignments: Δ_{xQ1}, Δ_{yQ1}, Δ_{xQ2}, Δ_{yQ2}, Δ_{xQ3}, Δ_{yQ3}) trainable alongside the standard GP hyperparameters, represented by equation 2.8. As more data is collected the learned ϕ gets closer to the true values so the physics model becomes more accurate eventually leading to better prediction and optimization performance.

$$(\phi^*, \omega^*) = \arg\max_{\phi,\omega} \log p(y|X, \phi, \omega) \tag{2.8}$$

Thus here, y are the observed objective values, X are input points evaluated so far and ϕ are the physics model parameters. By optimizing $\log p(y|X, \phi, \omega)$, it measures how well the model explains the observed data. Note that ω is always optimized during standard GP fitting, while ϕ is only optimized when explicitely set as trainable. This approach allows the physics model to adapt better and gradually converge towards their true values, making the prior more accurate and speeding up the convergence by mitigating the mismatch between the model and reality.

2.4 Cheetah Based Differential Simulation Library

Using physics knowledge based simulator as a prior mean function requires both speed (so the simulator must be fast enough to call within each GP evaluation) and differentiability (in order to facilitate gradient based hyperparameter optimization). Cheetah [4], was developed to satisfy these needs. Cheetah makes a conscious trade-off, in order to achieve speed, it gives up some fidelity. Its main objective is to give machine learning applications a differentiable beam dynamics code with improved speed over existing simulation codes. This ability of differentiation in Cheetah is important for our approach because when GP fits its hyperparameter by maximising marginal likelihood, gradients flow through the cheetah simulation to update the trainable misalignment parameter. Gradients of beam dynamics are often expensive to compute analytically or numerically [4] but cheetah overcomes this by utilizing *PyTorch* automatic differentiation. So, instead of manually deriving gradients or approximating them numerically, which would then require multiple function calls per parameter, *PyTorch* tracks operations as they happen and can compute exact gradients autometically through backpropagation. Aditionally, this enables BO packages like *BoTorch* to optimize acquisition functions effectively. Cheetah provides two beam representations.

- **Parameter Beam:** assumes a Gaussian beam and represents it using a seven dimensional mean vector and covariance matrix, only these statistical moments are propagated through the lattice, enabling really fast simulation.
- **Particle Beam:** represents the beam as individual macro-particles, each decribed by seven dimenional vector. This allows modeling of non-Gaussian distribution and bunch substructures, but at greater computational cost.

2.5 The ARES accelerator

The Accelerator Research Experiment at SINBAD (ARES) facility at DESY serves as a testbed for advanced accelerator concepts and to test machine learning techniques on them. It is an S-band radio frequency linac with two independently powered travelling wav accelerating strcutures and a photoinjector. These structures can operate at energies upto 154 MeV. ARES's main goal is to create and investigate sub-femtosecond electron bunches at accelerated energy. For uses like radiation production by free leectron lasers, the capacity to produce such brief bunches is highly desirable. ARES is also used for medical applications and for the study and development of accelerator components [3].

The project work focuses on the ARES experimental area. The beamline consists of three quadrupole magnets (Q1, Q2, Q3) and two corrector/steering magnets (CV for vertical steering and CH for horizontal steering), and a diagnostic screen where beam properties are measured. The magnets sequence of ARES Experimental Area, which is a subsection of ARES is depicted in figure 2.2 where two quadrupole magnets (Q1 and Q2) are followed by the vertical corrector magnet (CV), then the third quadrupole magnet (Q3) and finally the horizontal corrector magnet (CH) before reaching the screen. Each magnet is separated by a drift space (d_1, d_2, d_3, d_4, d_5, d_6) each having lengths within the range of 0.175 to 0.450 meters.

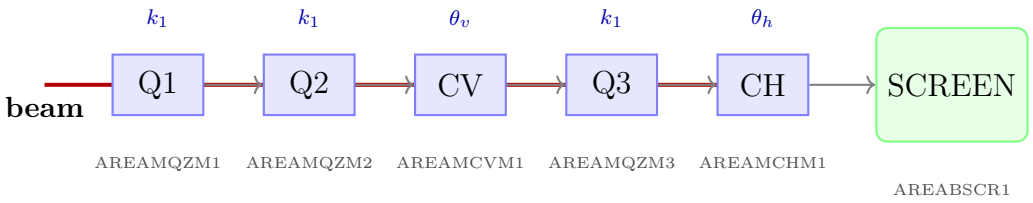

Figure 2.2: Schematic of the ARES Experimental Area showing the magnet sequence.

All magnets have polarity-switchable power supplies. It is possible to actuate the quadrupole magnets upto a 72 m field strength. The steering/corrector magnets have a limit of 6.2 mrad. The camera's field of view is approximately 8 mm by 5 mm, with a pixel resolution of 2448 by 2040. The scintillating screen's effective resolution is about 20 μm [3]. The beam tuning task involves optimizing these five magnets:

$$\mathbf{u} = (k_{Q1}, k_{Q2}, \theta_{CV}, k_{Q3}, \theta_{CH}) \tag{2.9}$$

Where in equation 2.9, $\mathbf{u} \in \mathbb{R}^5$. In order to produce a focused and centred beam at screen, The measured observables are beam's horizontal and vertical positions (μ_x and μ_y) and sizes (σ_x and σ_y). The objective function (equation 2.10) used throughout this work is the mean absolute error which combines both centering and focusing of the beam into a single metric to minimise. Where σ_x and σ_y are strictly positive since they are beam size.

$$mae = \frac{1}{4}(|\mu_x| + \sigma_x + |\mu_y| + \sigma_y) \tag{2.10}$$

In a real world accelerator, the quadrupole magnets are never exactly aligned; the precise beam distribution entering the experimental area is not known. Even with the corrector

magnets set to zero, a misaligned quadrupole with transverse offset (Δ_x, Δ_y) partially functions as steering element eventually diverting the beam. These misalignments are not directly measurable during operation. By treating the six misalignments parameters (Δ_{xQ1}, Δ_{yQ1}, Δ_{xQ2}, Δ_{yQ2}, Δ_{xQ3}, Δ_{yQ3}) as trainable quantities within the physics-informed prior, our approach addresses this issue too for the prior mismatched scenario and allows the model to adjust to actual accelerator settings using the optimization data.

3 Implementation

This chapter covers the implementation of Bayesian optimization with a Cheetah based physics informed prior for the transverse beam tuning at the ARES Experimental Area. Instead of being one large script, the implementation code is systematically divided into python modules, each having their distinct functions:

- **`bo_cheetah_prior_ares.py`** includes the simulated ARES environment `ares_problem()` and the GPyTorch compatible prior mean class `ARESPriorMean` with its trainable misalignment parameters. This is where most of the physics lives.
- **`eval_ares.py`** constructs the optimizers (Standard BO, BO with cheetah prior and Nelder-Mead) along with the experimental scenarios (matched, mismatched and matched_prior_newtask) and loops through the optimization trials.
- **`eval_ares_metrics.py`** In accordance with the evaluation process [3], calculates the performance metrics used to compare the techniques.
- **`plot_ares_results.py`** generates plots for the results.

The plotting and metrics modules are then connected by the fifth script **`run_evaluation.py`**, which can also generate result tables in LaTex format. Figure (flow chart) provides an overview of how these pieces work together at runtime. Table abc shows the summary of primary libraries used.

Libraries	**Purpose**	**Version**
`Cheetah`	PyTorch based differentiable beam tracking simulator.	0.8.0
`GPyTorch`	Gaussian process framework, base class for prior mean module	1.14.2
`xopt`	BO loop, UCB acquisition Nelder-Mead baseline.	2.6.7
`botorch`	BO primitives used internally by Xopt	0.16.0

Table 1: Primary libraries/frameworks used.

3.1 Initial Setup

Before running any optimizers, we need something to optimize against. Defined in `bo_cheetah_prior_ares.py`, the method `ares_problem()` fulfills this purpose. A beam is propagated across a Cheetah model of the ARES experimental area using a set of magnet parameters, and the beam quality is then returned. In practice, every call to this function mimics what would happen if the user adjusted the magnet settings on the real machine and then looked at the diagnostic screen.

In this project, cheetah's ParameterBeam is used to describe the beam entering the experimental area. ParameterBeam was used simply because it is faster, which enables it to run the experiments more quickly and iterate on the implementation with shorter turnaround times. Also note that Cheetah's ParticleBeam could have also been just as well but with more computational cost. Since speed is not the most critical concern in this project, there is nothing holding us back from using either representation for this application. The ARES experimental area only involves linear optics, so both approaches get equivalent results. These values match the typical ARES operating conditions, with a beam energy of around 107 MeV and transverse sizes of 200 micrometres

A JSON file describing the full ARES accelerator is used to load the lattice itself. The coe extracts the sub-segment between AREASOLA1 (marker) and AREABSCR1 (screen) as we only require the experimental ares. Listing (code snippet- VOCS) demonstrates the loading of the lattice and the assignment of the five magnet parameters prior to tracking.

Listing 1: Code snippet: ARES lattice Loading and magnet parameters configuration.

```
1
2 # Load ARES lattice
3 ares_segment = cheetah.Segment.from_lattice_json("
    ARESlatticeStage3v1_9.json")
4 ares_ea = ares_segment.subcell("AREASOLA1", "AREABSCR1")
5
6 # Set magnet parameters
7 ares_ea.AREAMQZM1.k1 = torch.tensor(input_param["q1"])
8 ares_ea.AREAMQZM2.k1 = torch.tensor(input_param["q2"])
9 ares_ea.AREAMCVM1.angle = torch.tensor(input_param["cv"])
10 ares_ea.AREAMQZM3.k1 = torch.tensor(input_param["q3"])
11 ares_ea.AREAMCHM1.angle = torch.tensor(input_param["ch"])
12
13 if misalignment_config is not None:
14     for magnet_name , (dx, dy) in misalignment_config.items():
15         magnet = getattr(ares_ea, magnet_name)
16         magnet.misalignment = torch.tensor([dx, dy], dtype=
              torch.float32
17         )
18
19 out_beam = ares_ea(incoming_beam)
```

For the three quadrupole strengths (k^{Q1}, k^{Q2}, k^{Q3}) and corrector angles (θ^{CV} and θ^{CH}), we have limited the range from physical magnet's $\pm 72 m^{-2}$ to our simulation"s $\pm 30 m^{-2}$ and from $\pm 6.2 mrad$ to $\pm 6.0 mrad$ respectively because it does not make sense since there is no reasonable way for them to go that far out. The code iterates through the dictionary and applies each (Δ_x, Δ_y) pair to the corresponding quadrupoles when a misaligned configuration is provided. This enables us to model the realistic scenario in which magnets are not perfectly centred on the beam. To let the optimizer know how good or bad the result is, we need a single number. Following the previous ARES studies [4] approach, we

use the mean absolute error for each of the four beams observable.

$$mae = 0.25(|\mu_x| + \sigma_x + |\mu_y| + \sigma_y)$$

Here μ_x and μ_y are the beam positions (ideally, zero means beam is centred) and σ_x and σ_y are beam sizes (the smaller the better). Combining all four into a single metric forces the optimiser to center and focus the beam simultaneously. The implementation is straight forward. The function also returns the individual beam parameters in addition to MAE so that they can be logged for later analysis.

3.2 Physics Informed Prior Mean Module

The heart of this project work is the `AresPriorMean` class,defined in `bo_cheetah_ares.py`. Its role is to turn the cheetah simulation into a format that GPyTorch can use as the mean function of the Gaussian Process. Instead of using GP prior (typically with zero mean assumption), it begins with the best estimates from the physics model and just needs to learn the residual which is the difference between what the simulator predicts and what is actually observed.

`AresPriorMean` specifically inherits from `gpytorch.means.Mean` and implements a `forward()` method. Everytime GPyTorch needs the prior mean at a set of input points, it calls this method. The remaining GP components, such as kernel evaluation, posterior conditioning and acquisition function optimization, perform exactly like they would with any other mean function. Another important aspect is that Xopt builds the input tensor for the GP by sorting variables in alphabetical order, which means that since our magnets variables are named $q1$, $q2$, cv, $q3$ and ch, column 0 of the tensor would always be ch not $q1$, as one might expect from the beamline order. In order to avoid silently feeding the wrong values to incorrect magnets, the class defines explicit index constants.

Listing 2: Code snippet: Explicit indexing to ensure correct mapping in Xopt input tensor.

```

# Load ARES lattice
class AresPriorMean(Mean):
    # Considering alphabetical order
    VAR_ORDER = ['ch', 'cv', 'q1', 'q2', 'q3']
    IDX_CH = 0
    IDX_CV = 1
    IDX_Q1 = 2
    IDX_Q2 = 3
    IDX_Q3 = 4
```

As discussed earlier, in real accelerators, the quadrupole magnets are never perfectly aligned. A magnet that is even a fraction of a millimeter, diverting the beam in a way that would not be predicted by a perfectly aligned simulation, partially acting as a steering factor. In order to handle this, the prior module treats the quadrupole misalignments as learnable parameters (Δ_{xQ1}, Δ_{yQ1}, Δ_{xQ2}, Δ_{yQ2}, Δ_{xQ3}, Δ_{yQ3}) Each of the

three quadrupoles has horizontal and vertical misalignment values, making a total of six values. In order to do online system identification, GPyTorch's marginal likelihood maximization adjusts these values in addition to the standard GP hyperparameters as the optimizer collects data. Three coordinated layers are used in GPyTorch to register each misalignment parameter, these three layers are summarized in table 3.3. On top of this, @*property* accessors offers a convenient interface, reading a misalignment property applies the forward transform (raw to constrained) and when it is written, the inverse transform is applied. This three layer pattern is repeated identically for each of the six parameters. The constructor is fairly lengthy because of this six times repeated pattern, although a loop based factory would have been cleaner, but we go with the explicit version since it has the advantage of being easy to read and to debug.

Component	**GPyTorch API**	**What it does**
Raw Parameter	`register_parameter()`	A raw unconstrained `nn.Parameter` is created at first, which is what PyTorch's optimiser updates directly.
Interval Constraint	`register_constraint()`	Secondly, an interval constraint is used that maps this raw value through a sigmoid based transformation into the physically meaningful range of $\pm 0.5mm$, this serves as the bound guaranteeing that the learnt offset can never be greater than what is practical for quadrupole misalignments.
Smoothed box prior	`register_prior()`	Third, a `SmoothBoxPrior` is registered across the confined space, adding a soft regularization on top of the hard constraint. This helps the optimizer avoid getting stuck at the boundaries by gently penalizing values near the edges while assigning a fairly uniform probability throughout the interval

Table 2: Primary libraries/frameworks used.

When GPyTorch calls `forward()`, the method recieves a tensor X whose last dimension contains 5 columns (the five magnets, in alphabetical order), The steps include:

1. Using the index constants from Listing 3.4, slice out each magnet parameter.
2. Assign them to corresponding components of cheetah lattice
3. Apply the current misalignments values to the three quadrupoles by stacking them
4. Run the beam through the lattice
5. Return the MAE (Equation 3.1)

Since all of these operations (the beam tracking, the absolute values, and the summation) are native PyTorch operations, the full forward pass is differentiable. Gradients could flow backward through the simulator and into the six misalignment parameters, which is what allows the joint optimization of both misalignment values as well as usual GP hyperparameters

3.3 Optimization Runner and Evaluation Metrics

After setting up the environment and the prior setup, the remaining part is the runner that actually executes the experiments. This is handled by an `eval_ares.py` file that takes arguments for the experimental scenarios (matched, mismatched, matched_prior_newtask), the number of trials, the evaluation budget and the type of optimizer (BO, BO_prior, NM) where Bo is the standard Bayesian Optimization, bo_prior is Bayesian Optimization with physics prior and nm is Nelder-Mead. Xopt defines the optimization problem declaratively using a YAML based specification known as VOCS (Variable and Objective Configuration Specification). Listing 3.8 shows the five variables and the single objective. where the unit for quadrupole magnets ranges are m^{-2}, for corrector magnets ranges are rad and for objective (mae) is metre(m)

Listing 3: VOCS configuration.

```
vocs_config = """
        variables:
            q1: [-30, 30]
            q2: [-30, 30]
            cv: [-0.006, 0.006]
            q3: [-30, 30]
            ch: [-0.006, 0.006]
        objectives:
            mae: minimize
    """
    vocs = VOCS.from_yaml(vocs_config)
```

To test the prior under various conditions, the runner supports three scenarios given below, they are also summarized in Table 3.3:

1. **Matched Scenario:** This is the most simple and ideal scenario where the beam and misalignments in the environment are exactly what the prior expects, the default values (including all misalignment values to be zero), thus the physics model perfectly matches reality. This scenario exists mainly to verify that the implementation works correctly and to see how much the prior helps when it is perfectly accurate since there is no model reality gap at all. In practice, this situation never happens on a real machine.

2. **Mismatched Scenario:** In this scenario, a gap is introduced on purpose to create mismatch between the prior and the reality, a different beam (σ_y changed from

200μm to 100μm), and non zero misalignment values over all three quadrupoles are used in the environment(table 3.4). The prior, however, starts from the default beam and zero misalignments as it does not know about any of these changes. We tell Xopt to include the six misalignment parameters in the GP's marginal likelihood optimisation by setting `trainable_mean_keys=[\mae"]` in the StandardModelConstructor so they can be learned from data as it is received. This is the scenario that tests whether online systems identification actually works.

Magnet	Δ_x **(mm)**	Δ_y **(mm)**
AREASMQZM1	0.000	+0.200
AREASMQZM2	+0.100	-0.300
AREASMQZM3	-0.100	+0.150

Table 3: Ground truth quadrupolemisalignments

3. **Matched_prior_newtask Scenario:** The simulated accelerator setup is the same as in the mismatched scenario (modified beam and non-zero misalignments), but this time the prior is initialised with the correct values which are the right beam parameters and the true misalignment value. So this way we are hanging the prior a perfectly adjusted physics model and asking how fast can the optimizer converge when it already knows the true system's state, compared to having to learn it during the process.

For the evaluation in this project, the matched_prior_newtask serves as our actual matched case, since it uses the realistic accelerator setup having modified beam and misalignments rather than the ideal zero misalignment defaults. We compare four methods, Nelder-Mead, Standard BO (zero mean), BO with Cheetah prior (mismatched scenario), BO with Cheetah prior (matched_prior_newtask scenario). All four approaches face the same simulated accelerator so that the comparison is fair, what differs between them is only the optimization strategy, and in case of prior informed methods, how much the prior knowledge is available from the start for each method.

Scenario	**Modified Beam**	**True Misalignments**	**Prior Misalignment**	**Trainable** ϕ
Matched	—	—	—	—
Mismatched	✓	✓	0	✓
Matched prior (new task)	✓	✓	= true	—

Table 4: Three experimental scenario summary.

In order to have a fair comparisons, every trial strats from the same initial points:

$$x_0 = (k^{Q1} = 10.0, k^{Q2} = -10.0, \theta^{CV} = 0.0, k^{Q3} = 10.0, \theta^{CH} = 0.0) \tag{3.1}$$

After this initial evaluation, the runner loops through the `xopt.step()` for a fixed number of interactions. The default in the code is 50 steps, but for the evaluation presented in this report, we used 100 steps to give each method more room to converge and to better observe the later stage behaviors of the optimizers. Each step involves fitting the GP to all the data collected so far, selecting the next point by optimizing the UCB acquisition function and testing that point on the simulated accelerator. In the case of Nelder-Mead, the same `step()` interface updates the simplex instead. For mismatched scenario when the prior's misalignment parameters are trainable, fitting the GP also involves optimizing over ϕ. As a result, each new data point gives the marginal likelihood optimiser more information about where the magnets are actually positioned and the prior gradually gets more accurate which is the online system identification already discussed above. After each trial completes, the script stores the full MAE history, the overall best, and the learned misalignment values (where applicable). With a fixed random seed, ten separate trials are conducted per configuration and the results are saved to CSV files for further analysis.

3.4 Evaluation Metrics

The evaluation code is located in `eval_ares_metrics.py` that follows to [3]. The metrics computed for every trial are as follows:

1. **Final error:** is the best (minimum) MAE reached at any point during the trial
2. **RMSE:** is the root mean square of the overall best MAE curve that captures how quickly the optimizer improves and not just the final value
3. **Improvement percentage(%):** is the improvement measured from initial to best $(((\text{MAE}_0 - \text{MAE}_{\text{best}})/\text{MAE}_0) \times 100)$
4. **Steps to target:** shows how many evaluation does it takes to drop below 40 μm of MAE (which is the threshold set) for the first time
5. **Success rate:** is the percentage of trials that reach the 40 μm target within the budget
6. **Steps to convergence:** shows the first step after which the overall best varies by less than 10^{-6}

Though all the metrics provide quite good insights, the final error (the best MAE achieved) is the most crucial one for the practical purposes, and the steps to target are also informative because it tells how quickly the optimiser gets the beam into an acceptable state. `plot_ares_results.py` produces plots and tables, styled to match in [4] and [3], which can be seen under the results sections.

4 Result Discussion and Conclusion

This work offers an introduction on how to write a successful bachelor or matser's thesis at the Institute of Control Systems.

Surely, not all your questions have been answered. Ask your supervisor and use the opportunity that he corrects a chapter before handing in your final thesis, to improve your scientific writing!

References

[1] Deutsches Elektronen-Synchrotron DESY. "Desy - homepage", Accessed: Feb. 9, 2026. [Online]. Available: https://desy.de/

[2] J. A. Nelder and R. Mead, "A simplex method for function minization", *The Computer Journal*, vol. 7, no. 4, 1965.

[3] J. Kaiser et al., "Reinforcement learning-trained optimisers and bayesian optimization for online particle accelerator tuning", vol. 14, 2024.

[4] J. Kaiser, C. Xu, A. Eichler, and A. Santamaria Garcia, "Bridging the gap between machine learning and particle accelerator physics with high-speed, differential simulations", *Physical Review Accelerators and Beams*, vol. 27, 2024.

[5] T. Boltz et al., "Leveraging prior mean modules for faster bayesian optimization of particle accelerators", *Scientific Reports*, vol. 15, 2025.

[6] R. Roussel et al., "Bayesian optimization algorithms for accelerator physics", *Physical Review Accelerators and Beams*, vol. 27, 2024.

[7] S. Jalas et al., "Bayesian optimization of a laser-plasma accelerator", *Phys. Rev. Lett.*, vol. 126, p. 104 801, 10 Mar. 2021. DOI: 10.1103/PhysRevLett.126.104801 [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.126.104801

[8] J. Duris et al., "Bayesian optimization of a free-electron laser", *Phys. Rev. Lett.*, vol. 124, p. 124 801, 12 Mar. 2020. DOI: 10.1103/PhysRevLett.124.124801 [Online]. Available: https://link.aps.org/doi/10.1103/PhysRevLett.124.124801

[9] C. K. Williams and C. E. Rasmussen, *Gaussian processes for machine learning*. MIT press Cambridge, MA, 2006, vol. 2.